

```
class Participant:

    def __init__(self, fname, lname, email):
        self._fname = fname
        self._lname = lname
        self._email = email

    def name(self):
        return self._fname + ' ' + self._lname

    def email(self):
        return self._email
```

Kapitel 10

Objektorientering och arv

Arv är en central princip i objektorientering (OO) som handlar om att på ett strukturerat sätt kunna återanvända kod och funktionalitet i redan definierade klasser med möjligheten att *lägga till* eller *ändra* på den existerande klassen. Ofta så finns redan en klass som nästan gör det man vill, och det är då en god idé att låta *ärva* från den klassen.

Ett enkelt exempel är vi har en klass A som vi vill använda funktionaliteten från, men ändra på vissa delar:

```
class A:
    """This is a base class
    """
    class_integer = 10

    def __init__(self):
        self.vals = [5]

    def power_sum(self):
        return sum([i**2 for i in self.vals]) + A.class_integer

class B(A):
    """This is a sub class
    """
    def __init__(self, x, y):
        self.vals = [x, y]

    def print_result(self):
        print(self.power_sum())
```

Här är A en *basklass* (eng. *base class*) för B och B är *subklass* (eng. *sub class*) till A. Man kan också säga att A är *superklass* (eng. *super class*) till B. I flera programmeringsspråk används ofta de engelska termerna *parent class* och *child class* för bas- och subklass.

Lägg märke till att **class**-definitionen av B tar A som ett argument, vilket indikerar att B ärver från A. Med ett arv i detta fall så menas att A:s funktionalitet *automatiskt* kommer att vara implementerad i B,

så länge man inte aktivt definierar om något. I det här fallet har klass-konstruktorn för B, `__init__`, definierats om för att ta *två* istället för *ett* argument (x och y). Utöver detta så används funktionaliteten från A: t.ex. finns i ovan exempel inget skäl att definiera om `power_sum` i B.

Användningen är ganska rättfram:

```
a = A()
print(a.power_sum())    # method defined in A

b = B(10, 20)
print(b.power_sum())    # calling method in B inherited from A
b.print_result()        # uses method specific to B
```

```
35
510
510
```

Obs: i Python ärver man inte instansattribut automatiskt! Det görs i många andra programmeringsspråk.

```
class C(A):
    def __init__(self, y):
        self.vals.append(y)
```

```
c = C(10)
```

```
AttributeError: 'C' object has no attribute 'vals'
```

För att se till att C ärver även instansattribut från A, får vi ändra konstruktorn lite genom att anropa `super`, som ser till att metoden som följer anropas på i basklassen:

```
class C(A):
    def __init__(self, y):
        super().__init__() # call constructor of super class
        self.vals.append(y)

class D(B):
    def __init__(self, y):
        super().__init__(5, 10) # call constructor of super class
        self.vals.append(y)
```

```
c = C(10)
d = D(10)
print(c.power_sum())
d.print_result()
c.print_result()    # raises an Error
```

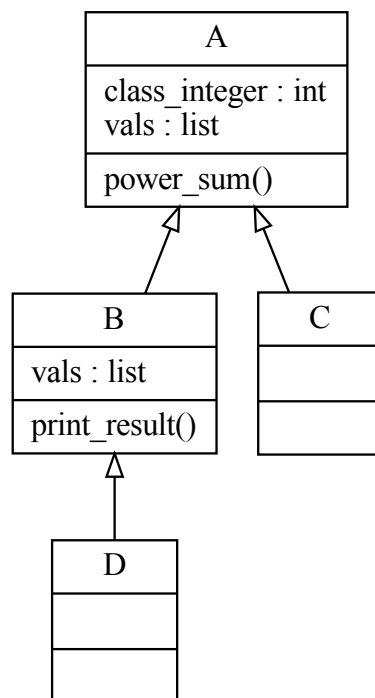
```
135
235
AttributeError: 'C' object has no attribute 'print_result'
```

Felet uppstår på grund av att `print_result()` inte är definierad för C, som ärvt från A. Klass D däremot har ärvt metoden från B.

10.1 Klassdiagram

Ett klassdiagram visualiserar hur klasser hänger ihop. Det innehåller ingen information om objekt. Man kan ange att en klass ärver från en annan (relationen “is-a”, dvs “är en”) och att en klass innehåller objekt från en annan klass (“aggregation” och “komposition”).

Det finns många sätt att rita klassdiagram och informell notation används ofta. Det finns professionella verktyg, som ofta bygger på industristandarden UML, som genererar klassdiagram från kod. T.ex. är fig. 10.1 genererad m.h.a. Python-biblioteket pyreverse (som är en del av kodanalysverktyget pylint).



Figur 10.1: Klassdiagram för A, B, C och D

Ett *objektdiagram* liknar till stor del ett klassdiagram, men förklarar *tillståndet* i ett program vid en viss tidpunkt: t.ex. skulle ett objektdiagram av ovan visa att `a.vals` innehåller `[5]`, `b.vals` innehåller `[10, 20]`, `c.vals` innehåller `[5, 10]` och `d.vals` innehåller `[5, 10, 10]`. Klassdiagrammet i fig. 10.1 förklarar relationer mellan klasser oavsett tidpunkt.

10.2 Substitutionsprincipen

Det är erkänt svårt att bestämma bra klasser. Många böcker har skrivits på ämnet, med många olika tillvägagångssätt för att bestämma vilken klass-struktur man ska ha. Det här kan bli väldigt komplicerat

fort, men en väldigt bra tumregel, som är känd som [Liskov's substitutionsprincip](#) (efter MIT professorn och Turingprisvinnaren Barbara Liskov), är:

- Allt som gäller för en klass ska gälla för dess subclasser.

Principen ger *god design*. Det kanske låter som en självklar princip, men det är lätt att vilja plocka bort funktionalitet i subclasser och det finns många exempel som bryter mot principen.

10.3 Funktioner som ger information om klasser

Två användbara inbyggda funktioner när det gäller klasser är `isinstance` och `issubclass`. Den första funktionen testar om ett givet *objekt* är en *instans* av en klass, och den andra om en *klass* är en subclass av en annan *klass*, dvs om en klass ärver från en annan.

Om vi använder oss av klassdefinitionerna (A, B, C, D) och instanserna (a, b, c, d) ovan, så kan vi t.ex. göra:

```
isinstance(a, A) # a is an instance of the class A
isinstance(d, A) # d is an instance a class that inherits from A
isinstance(d, C) # d is not an instance of a class that inherits from C

issubclass(A, A)
issubclass(D, A)
issubclass(D, C)
```

```
True
True
False

True
True
False
```

Lägg märke till att det inte bara är den *direkta* basklassen som kollas med funktionerna, utan även vidare tillbaka i "arvsledet".

I programmeringsspråk som stödjer multipelt arv (som Python, mer om det i kapitel 11) så kallas ett sådant "arvsled" *method resolution order* (förkortas vanligtvis som *MRO*). MRO specificerar i vilken ordning basklasserna (om flera) till en klass Python söker efter metoder. I Python har klasser en metod `.mro` som visar klassens MRO. T.ex:

```
A.mro()
D.mro()

[__main__.A, object]
[__main__.D, __main__.B, __main__.A, object]
```

Här kan vi se att D ärver från B (som är den första klassen efter D som söks för om en metod finns D) och vidare från A. Den sista elementet i listan, `object`, är den *fundamentala byggstenen* för typer i Python och är därför sist i listan och både A och D har detta som en del av sin MRO. När det gäller Python får man ofta höra "Everything is an object in Python":

```
isinstance(a, object)
isinstance(print, object) # function print is an object
issubclass(str, object)
issubclass(int, object)
```

```
True
True
True
True
```

10.4 Exempel: Geometriska figurer

Som ett exempel på arv ska vi nu implementera klasser som representerar geometriska objekt (figurer) som samlar vanliga/typiska operationer som man kan vilja göra på dessa figurer. Meningen är att med dessa klasser:

- Illustrera värdet av arv
- Illustrera *överlagring av metoder* (eng.: *method overriding/overloading*)
- Illustrera värdet av *inkapsling* (eng.: *encapsulation*, samling av data-attribut och de metoder som agerar på datat)
- Rita klassdiagram

Vi vill skriva kod som implementerar Python-objekt som representerar olika geometriska figurer som består av (raka) sidor. För att följa Liskovs substitutionsprincip, så vill vi ha en basklass som representerar det mest allmänna fallet: en polygon. Subklasserna ska lägga till data och funktionalitet som representerar en specifik typ av geometrisk figur.

Vi börjar med att definiera basklassen Polygon, som är menad att vara en klass som täcker det allmänna fallet men med vissa begränsningar.

```
class Polygon:

    __name__ = "Polygon"    # Klass attribut

    def __init__(self, n_sides, sides=None):
        self.n_sides = n_sides
        if sides:
            self.sides = sides
        else:
            self.sides = [1 for _ in range(n_sides)]
            # Standardvärde sätter alla sidor till längd 1
        self.check_sides()

    # "Private" methods
    def _get_side(self, side_no):
        s = float(input("Enter side " + str(side_no + 1) + ": "))
        return s

    # "Public" methods
    def check_sides(self):
```

```

    # Check that all sides have positive lengths
    if (not self.sides or
        not all(map(lambda x: x > 0, self.sides))):
        raise ValueError("Invalid side lengths")

    # Check that the number of sides are correct
    if self.n_sides != len(self.sides):
        raise ValueError(
            "Wrong number of sides specified in the sides list"
        )

    def set_sides_from_input(self, n_unique_sides = None):
        if n_unique_sides and self.n_sides % n_unique_sides:
            raise ValueError(
                "The total number of sides must be an integer"
                " multiple of the number of unique sides"
            )

        if not n_unique_sides:
            n_unique_sides = self.n_sides

        print("Please enter " + str(n_unique_sides) +
              " unique side lengths for a " + self.__name__)
        unique_sides = [self._get_side(i) for i in range(n_unique_sides)]
        self.sides = unique_sides * (self.n_sides // n_unique_sides)
        try:
            self.check_sides()
        except ValueError as e:
            print(e)
            self.set_sides_from_input() # Recursive call

    def display_sides(self):
        for i in range(self.n_sides):
            print("Length of side", i + 1, ":", self.sides[i])

```

Klasskonstruktorn tar två argument, ett obligatoriskt argument `n_sides` som specificerar hur många sidor polygonen skall ha och ett nyckelordsargument `sides` som har standardvärdet `None`. I konstruktorn så sätts instansattributen till de värden som de formella parameterarna har; om `sides` inte har specificerats så sätts instansattributet till en lista med `n_sides` stycken 1:or, dvs att alla sidor har längd 1. För att en polygon ska vara giltig måste några enkla krav vara uppfyllda, dessa kollas genom att anropa på hjälpmetoden `check_sides` som lyfter ett särfall om något krav inte är uppfyllt.

Det första kravet som kollas i `check_sides` är att alla sidor är positiva. Det görs med `map` och en anonym funktion (`lambda`) och om kravet inte är uppfyllt lyfts ett särfall. Den första delen av villkoret för `if`-satsen i `check_sides` kontrollerar att `sides` är satt, dvs om `sides == None` så exekveras satsen *oavsett* om det andra villkoret är uppfyllt eller ej. Denna sats är av värde då `map` skulle ge ett fel om `sides == None`, eftersom `None` inte går att iterera över. Genom att utvärdera `if not sides` först så exekveras `if`-satsen om `sides == None` (och `map` exekveras aldrig) *eller* så är `sides` satt (vi antar att det är en lista) och `map` kan iterera över `sides`. Det andra kravet som sedan kollas är att antalet sidlängder i listan `sides` faktiskt

stämmer överens med antalet sidor som polygonen ska ha (`n_sides`).

Huvudmetoden i `Polygon` är `set_sides_from_input`, som när den anropas först skriver ut ett meddelande och sedan med hjälp av en listomfattning läser in `n_unique_sides` antal sidlängder. Vi vill här ge möjligheten att kunna specificera att en polygon har `n_sides` *antal* sidor men bara har ett visst antal unika sidlängder, till exempel skulle `n_unique_sides=1` betyda att alla sidor i polygonen är lika långa. Vi vill dock begränsa hur en användare kan specificera antalet unika sidor i en polygon: det totala antalet sidor måste vara en heltalsmultipel av antalet unika sidor. Vi kollar att detta är uppfyllt genom att se om villkoret `n_sides % n_unique_sides` utvärderas till `True`, dvs om det finns en rest efter divisionen (ett tal skiljt från 0) så lyfter vi ett `ValueError` (kom ihåg att alla tal som är skilda från 0 utvärderas till `True` i Python). Inläsningen av *varje enskild* sidlängd görs av den privata metoden `_get_side`. Eftersom vi redan sett till att `n_unique_sides` har ett tillåtet värde, så kan vi multiplicera listan med (unika) inlästa sidlängder med `n_sides // n_unique_sides` för att slutligen få en lista som innehåller *alla* sidlängder (dvs `len(sides) == n_sides`): i fallet att vi har färre unika sidlängder än totalt antal sidlängder, så repeteras listan med de unika sidlängderna ett lämpligt antal gånger. Slutligen försäkras vi oss att alla sidlängder är tillåtna med den privata metoden `check_sides`. Om en `ValueError` lyfts i `check_sides` så printas felmeddelandet i `error` och `set_sides_from_input` anropas på nytt.

Metoden `display_sides` skriver ut alla sidlängder i `sides`.

Nu när vi har en basklass, så kan vi använda arv för att definiera subklasser som representerar specifika polygoner. Till exempel kan en triangel-typ representeras så här:

```
class Triangle(Polygon):

    __name__ = "Triangle"

    def __init__(self, sides=None):
        super().__init__(3, sides)

    def check_sides(self):
        a, b, c = sorted(self.sides)
        triangle_condition = c < (a + b)
        if not triangle_condition:
            raise ValueError(
                "The sum of the shortest sides must be greater than "
                "the length of the longest side in a "
                + self.__name__
            )
        super().check_sides() # Villkoren för en triangel är striktare än en polygon

    # method specific to triangles
    def get_area(self):
        a, b, c = sorted(self.sides)
        # calculate the semi-perimeter
        s = (a + b + c) / 2
        return (s * (s - a) * (s - b) * (s - c))**0.5
```

Här ser vi att `Triangle` ärver från `Polygon` och att vi i `Triangles` konstruktor anropar *basklassens* konstruktor med två argument (`3, sides`). Att vi explicit anropar konstruktorn med `super` gör att `Polygon`

instantieras med `n_sides=3`, `sides=sides` och att en instans av `Triangle` ärver både instansattribut och metoder från `Polygon`. Vi överlagrar klassattributet `__name__` så att namnet för klassinstansen ges som `Triangle`. Vi överlagrar även `check_sides` så att den kollar att sidlängderna kan sättas ihop till en riktig triangel utöver att kolla kraven som gäller för alla polygoner. Vi *utökar* basklassens funktionalitet genom att lägga till en metod `get_area` som beräknar en triangelns area.

På ett liknande sätt kan vi definiera en rektangel. Rektanglar har fyra sidor, och max två *unika* sidlängder. Det här löses enkelt genom att definiera en klass `Rectangle` som ärver från `Polygon` som anropar basklassens konstruktor med argument korrekta argument, och att sedan överlagra `check_sides` och `set_sides_from_input` så att vi inte kan ha mer än två unika sidlängder.

```
class Rectangle(Polygon):

    __name__ = 'Rectangle'

    def __init__(self, sides=None):
        super().__init__(4, sides)

    def check_sides(self):
        a, b, c, d = self.sides
        rectangle_condition = a == c and b == d
        if not rectangle_condition:
            raise ValueError(
                "Opposite sides must be of equal length in a "
                + self.__name__
            )
        super().check_sides()
        # Villkoren för en rektangel inkluderar villkoren för en polygon

    # method specific for rectangles
    def get_area(self):
        return self.sides[0] * self.sides[1]

    # method specific for rectangles
    def get_diagonal(self):
        return (self.sides[0]**2 + self.sides[1]**2)**0.5

    def set_sides_from_input(self, n_unique_sides=2):
        if n_unique_sides > 2: # En rektangel har 2 eller 1 unika sidlängder
            raise ValueError(
                "A " + self.__name__ + " can't have more than 2 unique side lengths"
            )
        return super().set_sides_from_input(n_unique_sides)
```

`Rectangle` definierar också metoder som är specifika för rektanglar: `get_area` och `get_diagonal`.

Slutligen definierar vi ett specialfall av en rektangel, nämligen en kvadrat `Square`. En kvadrat är en rektangel, så både area och diagonal beräknas på samma sätt som i `Rectangle`. Det är därför lämpligt att ärva funktionaliteten av `Rectangle`.

```

class Square(Rectangle):

    __name__ = 'Square'

    def __init__(self, sides=None):
        super().__init__(sides)

    def check_sides(self):
        a, b, c, d = self.sides
        square_condition = a == b == c == d
        if not square_condition:
            raise ValueError(
                "All sides must have equal length in a "
                + self.__name__
            )
        super().check_sides()
        # Villkoren för en kvadrat inkluderar villkoren för en rektangel

    def set_sides_from_input(self):
        return super().set_sides_from_input(n_unique_sides=1)
        # En kvadrat har exakt 1 unik sidlängd

    def get_corner_coordinates(self):
        s = self.sides[0]/2
        return ((-s, -s), (-s, s), (s, s), (s, -s))

```

Skillnaden mellan en kvadrat och en rektangel är att det bara finns *en unik* sidlängd. För att åstadkomma detta så överlagrar vi `set_sides_from_input` igen och den här gången sätter att `n_unique_sides` alltid är lika med ett. Vi överlagrar även `check_sides` och lägger till det striktare villkoret att alla sidor måste vara lika långa. Slutligen utökar vi kvadrat med en metod som returnerar koordinaterna för kvadratens hörn.

Man kan nu använda sig av ovan definierade klasser:

```

t = Triangle()
t.input_sides()
print('The area of the triangle is:', t.get_area())
t.display_sides()

r = Rectangle()
r.input_sides()
print('The area of the rectangle is:', r.get_area())
print('The diagonal of the rectangle is:', r.get_diagonal())

s = Square()
s.input_sides()
print('The area of the square is:', s.get_area())
print('The diagonal of the square is:', s.get_diagonal())
print('The coordinates of the square corners:\n',
      s.get_corner_coordinates())

```

```

Please enter the 3 unique side lengths for a Triangle
Enter side 1 : 1
Enter side 2 : 2.5
Enter side 3 : 2
The area of the triangle is: 0.9499177595981665
Length of side 1 : 1.0
Length of side 2 : 2.5
Length of side 3 : 2.0
Please enter the 2 unique side lengths for a Rectangle
Enter side 1 : -1
Enter side 2 : 2
Invalid side lengths.
Please provide positive side lengths.
Please enter the 2 unique side lengths for a Rectangle
Enter side 1 : 1.2
Enter side 2 : 1.9
The area of the rectangle is: 2.28
The diagonal of the rectangle is: 2.247220505424423
Please enter the 1 unique side lengths for a Square
Enter side 1 : 6
The area of the square is: 36.0
The diagonal of the square is: 8.48528137423857
The coordinates of the square corners are:
((-3.0, -3.0), (-3.0, 3.0), (3.0, 3.0), (3.0, -3.0))

```

10.4.1 Implementera jämförelser

Som vi gick igenom i kapitel 9, så kan man genom att använda sig av specifika privata metoder implementera jämförelser för sin egen typ (klass). Dessa metoder fungerar väldigt bra att ärva från en basclass när någon logisk jämförelse mellan objekt kan göras.

Antag t.ex. att vi vill implementera <, >, och == baserat på arean av ett objekt. För de geometriska figurerna Triangle, Rectangle och Square så kan vi ju genom get_area räkna ut arean av ett objekt. Vi kan lägga till nedan funktioner i Triangle och Rectangle:

```

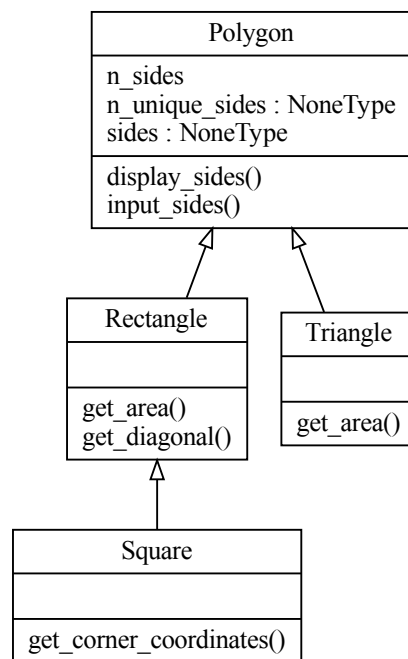
def __lt__(self, other):
    return self.get_area() < other.get_area()

def __gt__(self, other):
    return self.get_area() > other.get_area()

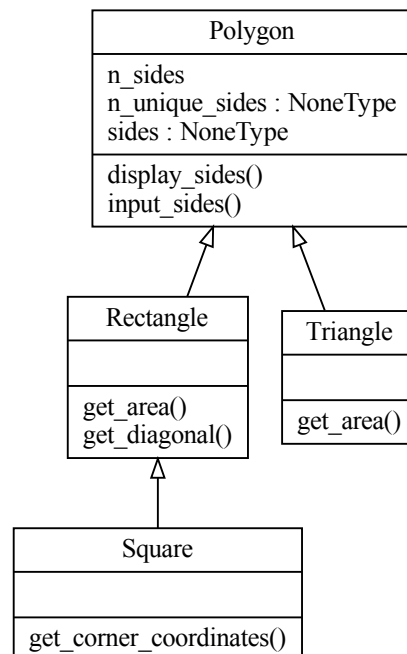
def __eq__(self, other):
    return self.get_area() == other.get_area()

```

Lägg märke till att vi *inte* behöver lägga till detta i Square, då denna klass ärver från Rectangle.



Figur 10.2: Klassdiagram för geometriska figurer.



I fig. 10.2 illustrerar vi klassdiagram för det här exemplet.

10.5 Exempel: Monster Go

Följande exempel är inspirerat av Pokemon Go och är ett lite större exempel. Exemplet är lite konstruerat, men det är för att framhäva viktiga poänger som illustrerar värdet av objekt-orienterad programmering.

Målet med detta exempel är att:

- Illustrera värdet av arv.
- Illustrera värdet av abstraktion: dölja implementationsdetaljer.

10.5.1 En basklass: ProgMon

Klassen ProgMon innehåller attribut och metoder som ska vara gemensamma för alla objekt. Vi försöker alltså även här följa Liskovs substitutionsprincip.

Vi kommer i följande exempel att använda oss mycket av privata metoder och attribut (både de som används av Python och börjar/slutar med `__`, t.ex. `__str__` och de som är privata för vår implementation och börjar med `_`, t.ex. instansattributen `_attack`).

```

class ProgMon:
    def __init__(self):
        self._attack = 0.0
        self._defense = 0.0
        self._caffeinated = False
  
```

```

        self._unit_testing = False

    def __str__(self):
        return "<ProgMon object>"

    def get_attack(self):
        if self._caffeinated:
            return 2 * self._attack
        else:
            return self._attack

    def get_defense(self):
        if self._unit_testing:
            return 2 * self._defense
        else:
            return self._defense

    def fight(self, other_progmon):
        """
        Attack is the best form of defense!
        Returns (True, False) if self wins, (False, True) if other_progmon
        wins and (False, False) otherwise.
        """
        if self.get_attack() > other_progmon.get_defense():
            return (True, False)
        elif other_progmon.get_attack() > self.get_defense():
            return (False, True)
        else:
            return (False, False)

```

10.5.2 Tre subklasser

I subklasserna Hacker, Newbie, och Guru lägger vi in specialisering (data och beteende som är specifika för klassen). Vi gör detta genom att ändra värden för de privata attributen och definiera `__str__`.

```

class Hacker(ProgMon):
    def __init__(self):
        super().__init__() # Call the superclass constructor!
        self._attack = 0.5
        self._defense = 0.25

    def __str__(self):
        return "<Hacker A=" + str(self._attack) + ">"

class Newbie(ProgMon):
    def __init__(self):
        super().__init__()
        self._attack = 0.15
        self._defense = 0.1

```

```

def __str__(self):
    return "<Newbie>"

class Guru(ProgMon):
    def __init__(self):
        super().__init__()
        self._attack = 1.0
        self._defense = 1.0

```

Provkör med:

```

h = Hacker()
n = Newbie()
g = Guru()
print(n.fight(h))
print(h.fight(n))
print(g.fight(h))

```

```

(False, True)
(True, False)
(True, False)

```

Obs: Gemensam funktionalitet i basklassen gör att vi undviker duplicering av kod.

10.5.3 Anpassning

Antag att man inte gillar tanken på att en Guru försöker vinna kamper. Vi kan implementera en “Don’t win, don’t lose” för en Guru som initierar en “fight”, genom att definiera en ny version av Guru:

```

class Guru(ProgMon):
    def __init__(self):
        super().__init__()
        self._attack = 1.0
        self._defense = 1.0

    # method overloading
    def fight(self, other_progmon):
        '''
        Don't win, don't lose.
        '''
        # same type of return value as in the super class
        return (False, False)

```

Provkör med ny definition: `g.fight(h)` ger `(False, False)`.

Observera att vi ändrar beteendet hos en subclass genom att definiera om en metod i subclassen.

10.5.4 Abstraktion undviker detaljkunskap

Basklassen `ProgMon` definierar ett *gränssnitt* som ger en praktisk abstraktion: du behöver inte kunna några detaljer om basklassen eller dess subclasser om du använder gränssnittet.

10.5.5 Exempel: en hacker dojo

Antag att vi skapar en klass för en “Hacker Dojo”, där ProgMon:s samlas för att träna. Man ska kunna utmana en Hacker Dojo, vilket kräver att man vinner en kamp mot varje medlem för att kunna säga att man vunnit. Förlust mot en medlem ger förlust av utmaningen.

```
class HackerDojo:
    def __init__(self):
        self._members = []

    def add_member(self, m):
        self._members.append(m)

    def challenge(self, pm):
        for monster in self._members:
            win, loose = pm.fight(monster)
            if loose:
                # Lost against one member, challenge failed.
                return False
        # Won against all members of the dojo
        return True
```

Upprovning:

```
dojo = HackerDojo()
dojo.add_member(h)
dojo.add_member(n)
print(dojo.challenge(g))
print(dojo.challenge(n))
```

```
True
False
```

Här vinner dojon mot gurun eftersom vår guru varken vinner eller förlorar (se den omdefinierade metoden `fight` i klassen `Guru`) och i metoden `challenge` så har vi programmerat att `HackerDojo` tycker att den vinner om den inte förlorar någon fjant.

Obs: det enda vi använder i implementationen är gränssnittet från `ProgMon`. Vi får automatiskt den lite annorlunda specialiseringen från `Guru`. Eller snarare: vi behöver inte oroa oss för sådana detaljer.

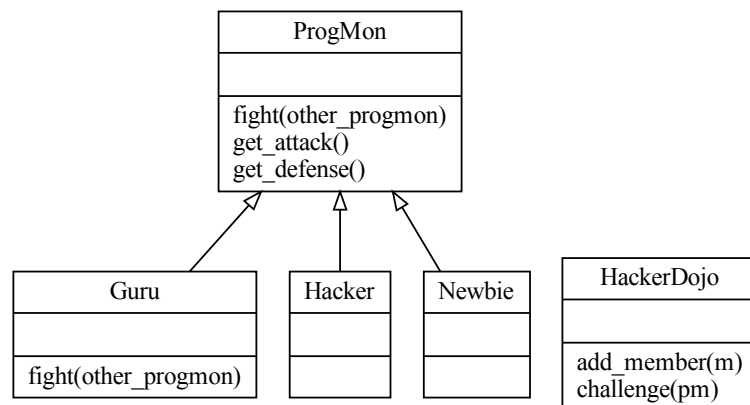
10.5.6 Ändrade detaljer

En av de stora vinsterna med objektorientering (men inte unikt för OO) är abstraktion. Genom att sätta ihop bra gränssnitt (dvs väl valda metoder) “framtidssäkrar” vi ett program: det blir lätt att ändra detaljer i en del av programmet, utan att andra delar påverkas. Detta underlättar ofantligt mycket i större kodprojekt som kan ha tusentals eller miljoner rader kod.

Antag att vi inser att `_attack` inte är ett bra attribut. Styrkan ges av erfarenhet, level, och andra attribut som blir specifika för subklasserna.

- Vilka ändringar behövs göras, och vilka klasser påverkas?
- Är `get_attack` en bra metod?

I fig. 10.3 illustrerar vi det här exemplet med ett klassdiagram.



Figur 10.3: Klassdiagram för Monster Go.

10.6 Uppgifter

1. Skapa en klass Equilateral som ärver från Triangle i exemplet ovan och metodöverlagrar `get_area`.

Tips: Arean för en liksidig triangel är $\sqrt{3}a^2/4$, där a är sidlängden. Man kan från standardmodulen `math` importera funktionen `sqrt`.

2. Skapa en klass Student som ärver från ProgMon (även instansattribut), men som lägger till ett instansattribut `is_learning` av typen `bool` och metodöverlagrar både `get_attack` och `get_defense` i ProgMon. Klassen Students metoder `get_attack` och `get_defense` skall använda sig av ProgMon `get_attack` och `get_defense` men ska multiplicera styrkan med två om Student lärs, dvs om `is_learning` är `True`. Du kan bestämma vad instansattributen `_attack` och `_defense` instansieras till i klassen själv.
3. I uppgift 5 och 6 i kapitel 9 implementerade vi en class Dog. Implementera en klass Tournament som

1. rankar hundarna baserat på antal tricks de kan, och
2. skriver ut resultaten.

Tips: hämta inspiration från hur HackerDojo relaterar till ProgMon.

4. Skriv en basklass Vehicle, som innehåller instansattributen `wheels`, `wings` och `sound`. Utöver detta ska Vehicles metod `__str__` returnera en sträng med information om fordonet (hur många hjul, etc., fordonet har). Skapa tre subclasser Car, Motorcycle och Plane som alla ärver från Vehicle, men har rätt antal hjul/vingar och gör "rätt" ljud.

Exempel-användning och output skulle vara:

```
car = Car()
mc = Motorcycle()
```

```
plane = Plane()
print(car)
print(mc)
print(plane)
```

```
A Car has:
4 wheels
and makes a AAARRRRRRRR! sound

A Motorcycle has:
2 wheels
and makes a VROOM VROOM! sound

A Plane has:
2 wings
and makes a WHOOSH! sound
```

5. Skriv en egen sträng-klass som implementerar ("hanterar") subtraktion och division. Med *subtrahera* en sträng menar vi här att ta bort en delsträng, och $s1 - s2$ innebär att den högraste förekomsten av $s2$ i $s1$ tas bort. Med *division* menas att alla förekomster av strängen till höger om $/$ operatoren tas bort från huvudsträngen, dvs $s1 / s2$ innebär att *alla* förekomster av $s2$ tas bort från $s1$.

Tips: skapa en subclass MyString till str, och läs på om vilka privata metoder som behövs implementeras för $-$ och $/$.

Exempel:

```
s1 = MyString("This is my own string!")
s2 = MyString("My, oh my, oh Oh.... oh")
op1 = MyString('my')
op2 = MyString('oh')
print("Removing first rightmost and then all instances of 'my' from s1:")
print(s1 - op1)
print(s1 / op1)
print("\nRemoving first rightmost and then all instances of 'oh' from s2:")
print(s2 - op2)
print(s2 / op2)
print("\nNo change in s1 expected here:")
print(s1 - 'hello')
print(s1 / 'hello')
```

```
Removing first rightmost and then all instances of 'my' from s1:
This is own string!
This is own string!
```

```
Removing first rightmost and then all instances of 'oh' from s2:
My, oh my, oh Oh....
My, my, Oh....
```

```
No change in s1 expected here:
```

```
This is my own string!  
This is my own string!
```